

1. Objetivo

Dibujar una imagen de tipo *bitmap*.

2. Ejercicio

Dibujar un tablero de ajedrez como el mostrado en la figura 2.1. utilizando un *kernel* bidimensional con bloques de 18×18 hilos y donde cada hilo se encargue de generar un píxel de la imagen final. Mostrar las propiedades del dispositivo seleccionado, las dimensiones del *kernel* lanzado y el tiempo de ejecución del *kernel*.

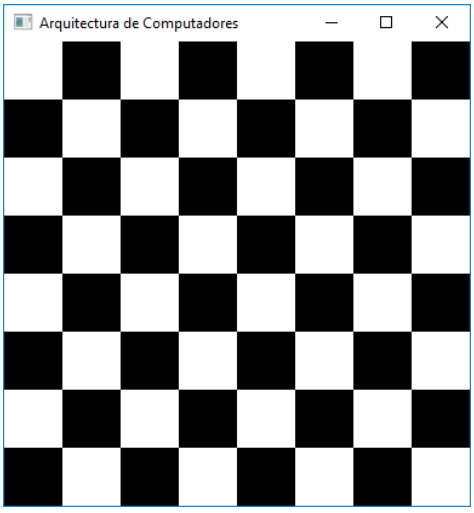


Figura 2.1. Tablero de ajedrez.

3. Elementos CUDA

Para realizar esta práctica se puede utilizar cualquiera de los recursos ya estudiados. Por otro lado, para poder visualizar el resultado y hacer que el Sistema Operativo nos muestre una ventana con la imagen del *bitmap* es necesario utilizar algunas funciones especiales. Las funciones que vamos a utilizar pertenecen a la API de **OpenGL** y están definidas en el archivo de cabecera "**`gpu_bitmap.h`**" que acompaña a este manual.

4. Conceptos teóricos

4.1. Imágenes digitales

Hay situaciones donde resulta interesante distribuir los hilos de ejecución de un *kernel* de CUDA de una manera análoga al problema que se pretende resolver, como es el caso de las operaciones con matrices. Esta situación se da también cuando estamos trabajando con imágenes digitales ya que, como veremos a continuación, una imagen digital es también una matriz de números.

Una imagen "real" es una función continua bidimensional $f(x, y)$ que representa el valor de una intensidad, generalmente luminosa, en función de las coordenadas espaciales x e y. Si convertimos estas cantidades continuas en cantidades discretas a través de los procesos

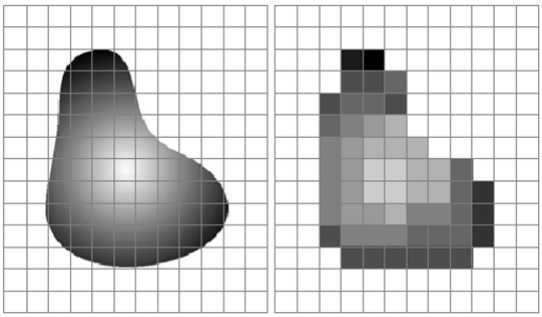


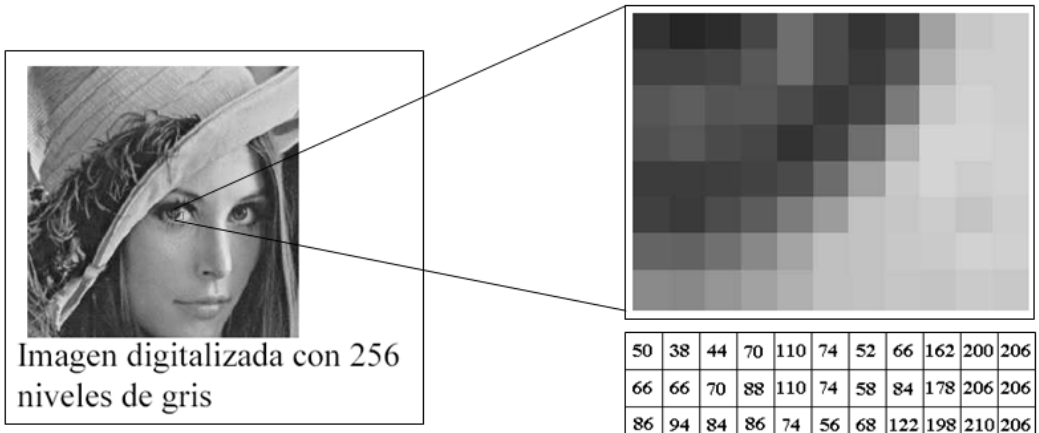
Figura 2.2. Imagen digital obtenida tras un proceso de muestreo y cuantificación.

denominados *muestreo* y *cuantificación* (Figura 2.2) obtenemos una imagen "digital":

- Cada muestra recibe el nombre de *elemento de imagen* o **píxel**.
- Cada valor de intensidad luminosa recibe el nombre de *nivel de gris*. El número de niveles de gris depende del número de bits empleados en el proceso de cuantificación. Por ejemplo, utilizando 8 bits el número de niveles disponibles es 256, desde el 0 (negro) hasta el 255 (blanco).

Tras el proceso de muestreo y cuantificación lo que obtenemos es una matriz de números donde cada número representa la intensidad luminosa en ese punto. Así, podemos decir que una imagen digital no es más que la representación de una imagen real a partir de una matriz numérica (Figura 2.3).

En conclusión, esto nos hace ver que la forma más adecuada para trabajar con imágenes digitales es utilizando un hardware que nos permita operar con grandes matrices de



50	38	44	70	110	74	52	66	162	200	206
66	66	70	88	110	74	58	84	178	206	206
86	94	84	86	74	56	68	122	198	210	206
80	88	74	70	50	66	110	176	212	212	210
60	60	62	66	74	108	160	200	212	206	210
64	58	76	92	124	156	194	198	204	196	206
100	98	112	136	164	192	194	200	202	210	208
138	136	150	162	176	192	194	198	196	202	200

Figura 2.3. Representación de una imagen digital como una matriz de números.

números. En este contexto, vemos que nuestro entorno de programación CUDA encaja perfectamente con dicho propósito ya que nos permite lanzar un *kernel* bidimimensional con hilos repartidos en ambas direcciones del espacio y haciendo corresponder a cada hilo con un píxel de nuestra imagen digital.

4.2. Imágenes en color

Desde el punto de vista de la información almacenada en una imagen digital podemos clasificar las imágenes digitales como:

- Imágenes de intensidad (o en escala de grises), donde cada elemento de la matriz representa un nivel de gris dentro del rango determinado por el número de bits empleado en la cuantificación (Figura 2.3).
- Imágenes en color, donde se necesita almacenar información de tres colores primarios definidos por el espacio de color que se esté utilizando.

El espacio de color que se utiliza para aplicaciones tales como la adquisición o generación de imágenes en color es el **RGB**, donde la combinación aditiva de los colores primarios rojo (*Red*), verde (*Green*) y azul (*Blue*) produce todo el rango de colores representables en dicho espacio (Figura 2.4).

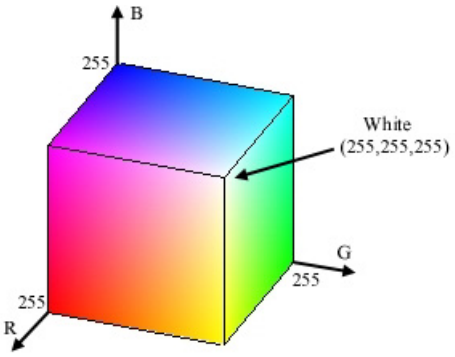


Figura 2.4. Representación del espacio de color **RGB** utilizando valores de 8 bits.

Una imagen en escala de grises también puede considerarse como una imagen en color en la que sus tres componentes son iguales. Así pues, para representar una imagen en color se necesita disponer de al menos tres valores por cada píxel, los correspondientes a las componentes roja, verde y azul.

4.3. Imágenes en mapa de bits (*bitmap*)

Un ***bitmap*** (o *mapa de bits*) es la forma natural de organizar en memoria la información de una imagen digital. Así, dado que una imagen digital se representa como una matriz rectangular de píxeles o puntos de color, en un mapa de bits la información de cada píxel se almacena en posiciones consecutivas de la memoria formando un array cuyo tamaño depende de la altura y la anchura de la imagen (número de píxeles) y de la información de color contenida en cada píxel (bits por píxel).

La información de color de cada píxel se codifica utilizando canales separados, cada uno de los cuales corresponde a uno de los colores primarios del espacio de color utilizado (generalmente **RGB**). A veces, se puede añadir otro canal que representa la transparencia del color respecto del fondo de la imagen y se denomina canal α (modelo **RGBA**). Por lo tanto, el tamaño necesario para almacenar en memoria un mapa de bits de una imagen en color de $M \times N$ píxeles, con 8 bits por canal y cuatro canales (R, G, B y α) es de $4 \times M \times N$ bytes. Y esta información se almacena en memoria principal ocupando posiciones consecutivas de memoria (Figura 2.5).

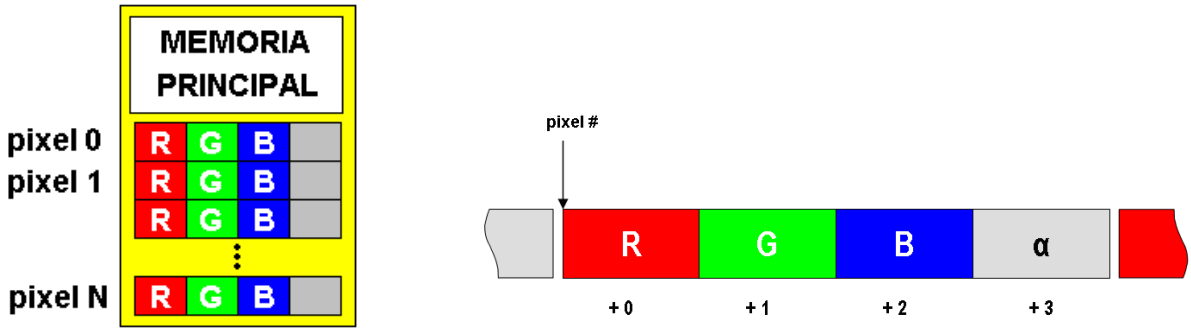


Figura 2.5. Ubicación en memoria principal de un mapa de bits de 4 canales y 8 bits por canal.

4.4. OpenGL

Con el fin de poder visualizar una imagen digital en nuestro sistema necesitamos funciones que hagan la labor de "abrir una ventana" y de "dibujar" en ella. La forma más sencilla es utilizar una **API** (*Application Programming Interface* - Interfaz de programación de aplicaciones) destinada a tal efecto como es **OpenGL**.

OpenGL (***Open Graphics Library***) es una especificación estándar, es decir, un documento que describe un conjunto de funciones y el comportamiento exacto que deben tener (partiendo de ella, los fabricantes de hardware pueden crear implementaciones). Consta de más de 250 funciones diferentes que pueden usarse para dibujar escenas tridimensionales complejas a partir de primitivas geométricas simples, tales como puntos, líneas y triángulos.

El funcionamiento básico de **OpenGL** consiste en aceptar primitivas tales como puntos, líneas y polígonos, y convertirlas en píxeles. Dado que está basado en procedimientos de bajo nivel, requiere que el programador dicte los pasos exactos necesarios para "renderizar" una escena. Esto contrasta con otras interfaces descriptivas, donde un programador sólo debe describir la escena y puede dejar que la biblioteca controle los detalles para representarla. El diseño de bajo nivel de **OpenGL** requiere que los programadores conozcan en profundidad la pipeline gráfica, a cambio de darles libertad para implementar algoritmos gráficos novedosos.

5. Ejemplo

El siguiente ejemplo muestra una forma muy sencilla de generar un *bitmap* y de mostrarlo por pantalla. Las llamadas a las funciones de **OpenGL** necesarias para su visualización están encapsuladas dentro del fichero de cabecera **`gpu_bitmap.h`** suministrado

junto a este manual. En este ejemplo se dibuja un *bitmap* donde cada hilo se encarga de dar color a un píxel asignando un número entero de 8 bits (un valor comprendido entre 0 y 255) para cada canal. En este ejemplo, para obtener un resultado "vistoso", el color de cada píxel se ha escogido asignando un valor relacionado con su posición espacial. El *bitmap* se dibuja en la pantalla de izquierda a derecha y de abajo a arriba, es decir, el píxel de coordenadas (0, 0) se sitúa en la esquina inferior izquierda.

```
// includes
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include "gpu_bitmap.h"

// defines
#define ANCHO 1024 // Dimension horizontal
#define ALTO   512 // Dimension vertical

// GLOBAL: funcion llamada desde el host y ejecutada en el device (kernel)
__global__ void kernel( unsigned char *imagen )
{
// ** Kernel bidimensional multibloque **
//
// coordenada horizontal de cada hilo
      int x = threadIdx.x + blockIdx.x * blockDim.x;
// coordenada vertical de cada hilo
      int y = threadIdx.y + blockIdx.y * blockDim.y;
// indice global de cada hilo (indice lineal para acceder a la memoria)
      int myID = x + y * blockDim.x * gridDim.x;
// cada hilo obtiene la posicion de su pixel
      int miPixel = myID * 4;
// cada hilo rellena los 4 canales de su pixel con un valor arbitrario
      imagen[miPixel + 0] = 255*x/( blockDim.x * gridDim.x);// canal R
      imagen[miPixel + 1] = 255*y/( blockDim.y * gridDim.y);// canal G
      imagen[miPixel + 2] = 128;                            // canal B
      imagen[miPixel + 3] = 0;                              // canal alfa
}
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
// Declaracion del bitmap:
// Inicializacion de la estructura RenderGPU
      RenderGPU foto(ANCHO, ALTO);

// Tamaño del bitmap en bytes
      size_t bmp_size = foto.image_size();

// Asignacion y reserva de la memoria en el host (framebuffer)
      unsigned char *host_bitmap = foto.get_ptr();

// Reserva en el device
      unsigned char *dev_bitmap;
      cudaMalloc( (void**)&dev_bitmap, bmp_size );

// Lanzamos un kernel bidimensional con bloques de 256 hilos (16x16)
      dim3 hilosB(16,16);
// Calculamos el numero de bloques necesario (un hilo por cada pixel)
      dim3 Nbloques(ANCHO/16, ALTO/16);
// Generamos el bitmap
      kernel<<<Nbloques,hilosB>>>( dev_bitmap );

// Copiamos los datos desde la GPU hasta el framebuffer para visualizarlos
      cudaMemcpy( host_bitmap, dev_bitmap, bmp_size, cudaMemcpyDeviceToHost );

// Visualizacion y salida
// La funcion "display_and_exit()" no retorna e impide continuar con el main()
      foto.display_and_exit();

      return 0;
}
```